A second option is to make the `data` pointer atomic and set that with a call to compare/exchange. If the call succeeds, this is your tail node, and you can safely set the `next` pointer to your new node and then update `tail`. If the compare/exchange fails because another thread has stored the data, you loop around, reread `tail`, and start again. If the atomic operations on `std::shared_ptr<>` are lock-free, you're home free. If not, you need an alternative. One possibility is to have `pop()` return a `std::unique_ptr<>` (after all, it's the only reference to the object) and store the data as a plain pointer in the queue. This would allow you to store it as a `std::atomic<T*>`, which would then support the necessary `compare_exchange_strong()` call. If you're using the reference-counting scheme from listing 7.11 to handle multiple threads in `pop()`,`push()` now looks like this.

Listing 7.14. A (broken) first attempt at revising `push()`

```
void push(T new_value)
{
    std::unique_ptr<T> new_data(new T(new_value));
    counted_node_ptr new_next;
    new_next.ptr=new node;
    new_next.external_count=1;
    for(;;)
    {
        node* const old_tail=tail.load();     <-1
        T* old_data=nullptr;
        if(old_tail->data.compare_exchange_strong(
            old_data,new_data.get()))                <-2
        {
            old_tail->next=new_next;
            tail.store(new_next.ptr);     <-3
            new_data.release();
            break;
        }
    }
}
```

Using the reference-counting scheme avoids this particular race, but it's not the only race in `push()`. If you look at the revised version of `push()` in listing 7.14, you'll see a pattern you saw in the stack: load an atomic pointer